

β

This project is currently under beta-test, until it is validated by a fair number of students or a decent delay has elapsed.

- *Please report any typo, incoherence, inconsistency, error, using the form <https://tally.so/r/3lVDJo>.*
- *A dedicated Slack channel exists to exchange on beta-test projects: [#_world_beta_test](#).*
- *A FAQ page dedicated to beta-test projects is available [here](#).*
- *The project currently awards 0 XP, on purpose; XP will be granted once the beta-test is over and conclusive.*





Datomic

A databases project

42 Staff pedago@42.fr

Summary: Query six families of databases, then learn to pick the right one and federate them instead of duplicating data.

Version: 1.0

Contents

I	Foreword	2
II	Introduction	3
III	General rules	4
III.1	The virtual machine	4
III.2	The engines	5
III.3	The playground	5
III.4	Reading a statement	5
III.5	Your data	6
IV	Mandatory part	7
IV.1	Module A, SQLite: SQL foundations	7
IV.2	Module B, PostgreSQL: the relational core	7
IV.3	Module C, Redis: key-value	7
IV.4	Module D, ClickHouse: columnar	7
IV.5	Module E, Neo4j: graph	8
IV.6	Module F, DuckDB: federation	8
V	Review	9
V.1	How it runs	9
V.2	What is graded	10

Chapter I

Foreword

Somewhere on this station, right now, a database is quietly refusing to lose your data. That is the entire job, and it is harder than it sounds. Ask a room of engineers to define the word “database” and half will say “a place to put things” while the other half start an argument about consistency that outlives the meeting.

There is no single database anyway, only a family of very opinionated ones. A relational engine treats every row like a signed contract and refuses to let you break it. A key-value store has exactly one talent, telling you what sits at a key, and it does that before you have finished asking. A columnar engine will read a billion rows and cheerfully skip every column you did not ask for. A graph engine thinks in relationships and finds the friend of a friend of a friend without sweating, right where a tall stack of joins would keel over. And NULL, sooner or later, will trip you: it is not zero and not empty, it is the database shrugging and saying “I have no idea”, and it never means yes.

Knowing the syntax of one of them is a start. Knowing which one to reach for, and how to make several of them answer a single question without copying data between them, is the actual skill. That is what this project is about. Welcome aboard.

Chapter II

Introduction

Datomic is a solo project built around a station: a self-contained virtual machine you boot once and work inside. It hosts a small web playground where you receive a problem, write a query, run it, and submit it for an automatic check.

You climb twenty levels. The first four are plain SQL on SQLite. The next nine build up the relational core on PostgreSQL, from joins to window functions, transactions, indexes, modelling and federation. Two levels cover Redis, two cover ClickHouse, two cover Neo4j, and the last one federates several engines in a single query.

Every level hands you one problem drawn from a bank of variants, so you are unlikely to be working on the same statement as the cadet next to you, and if you are, your data is still not theirs: names are shuffled per login while the structure stays the same for everyone. Copying a neighbour's query is a poor bet and copying their result is a worse one.

The point is never the syntax of six databases. It is judgement: recognising the shape of a problem, choosing the store that fits it, and federating instead of duplicating.

You discover the SQL yourself, level by level, in the browser playground. Nothing is handed in. The project is validated in a live review, where you sit an exam drawn from a bank you have never seen and explain your answers out loud.

Chapter III

General rules

III.1 The virtual machine

The project ships as a single OVA you import into VirtualBox. Everything runs inside it: the engines, the playground and your work. The machine needs no network once imported.



The image is an x86-64 guest, so it needs an x86-64 host: any campus iMac or PC, or an Intel Mac. On an Apple Silicon Mac there is no hardware virtualisation for x86-64, and emulating it in software does boot the VM but cannot keep six database engines answering, so do not go down that road. Work on a campus machine instead.

- Boot the VM. The playground starts on its own.
- From your host browser, open `http://localhost:8042`.
- Enter your intra login. Your levels and your data are derived from it.

The VM has no login and no shell. Its console only tells you how to reach the playground; everything you do happens in your browser.

```
Datonnie
42 databases station
by wil

This station has no shell. There is nothing to log into here.

Open the playground in your host machine's browser:

    http://localhost:8042

Sign in with your intra login, for example: wil
Your levels and your data are derived from it.

(You can close this window. The station keeps running in the background.)
```

Figure III.1: The VM console at boot: nothing to log into, just the playground URL.

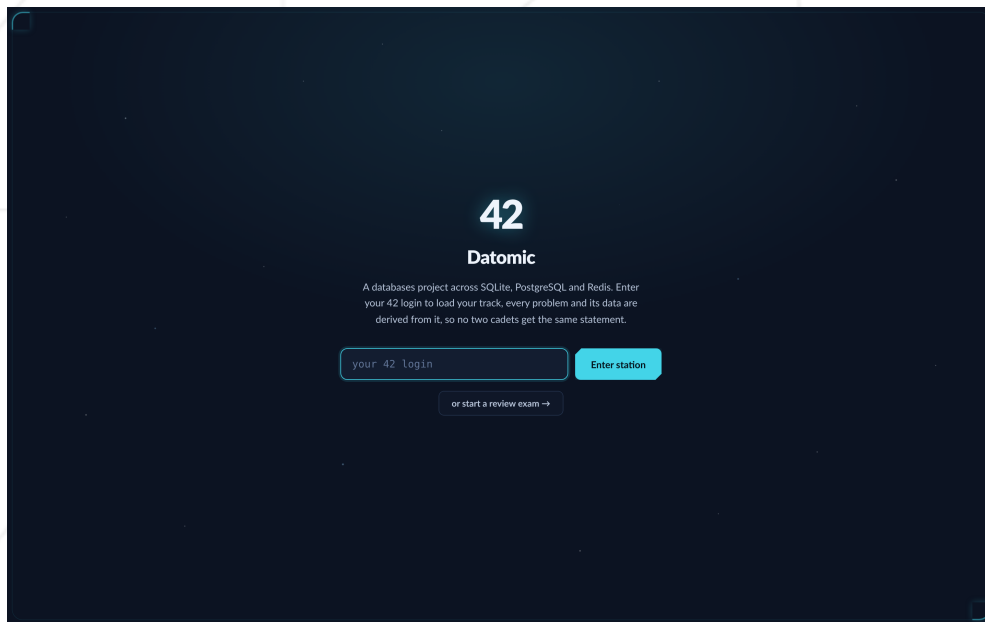


Figure III.2: The station gate. Enter your intra login to load your own levels.

III.2 The engines

Every engine starts on its own when the VM boots. There is nothing to launch and nothing to log into: you never touch a shell. ClickHouse and Neo4j take a few dozen seconds to load their data the first time, so if you reach one of those levels in the first minute after boot, give it a moment and reload.

III.3 The playground

For each level you get a statement, a query editor and a view of your own data shape. Running a query shows the rows. Submitting it grades the result against a hidden reference. Feedback on a wrong answer tells you the shape of the mistake, a row count or a column mismatch, never which values are wrong.

Some levels change your own database (creating a table, an index, a schema, a materialised view, a foreign server). On those, **Run** is allowed to write too, so you can build the thing and look at it before submitting, and a **Reset data** button puts your instance back to its original state whenever you want a clean slate.

Clearing one variant of a level opens the next. The bank stays open for as many extra passes as you want.

III.4 Reading a statement

A statement always tells you the shape of the answer: how many columns, which ones, and in what order. When a value has to be rounded, it says to how many decimals. When rows have to come back in a given order, it says so, and that order is graded; when it says nothing about order, your rows can come in any order. Anything the statement does not pin down is not graded, so you are free to name your columns as you like.

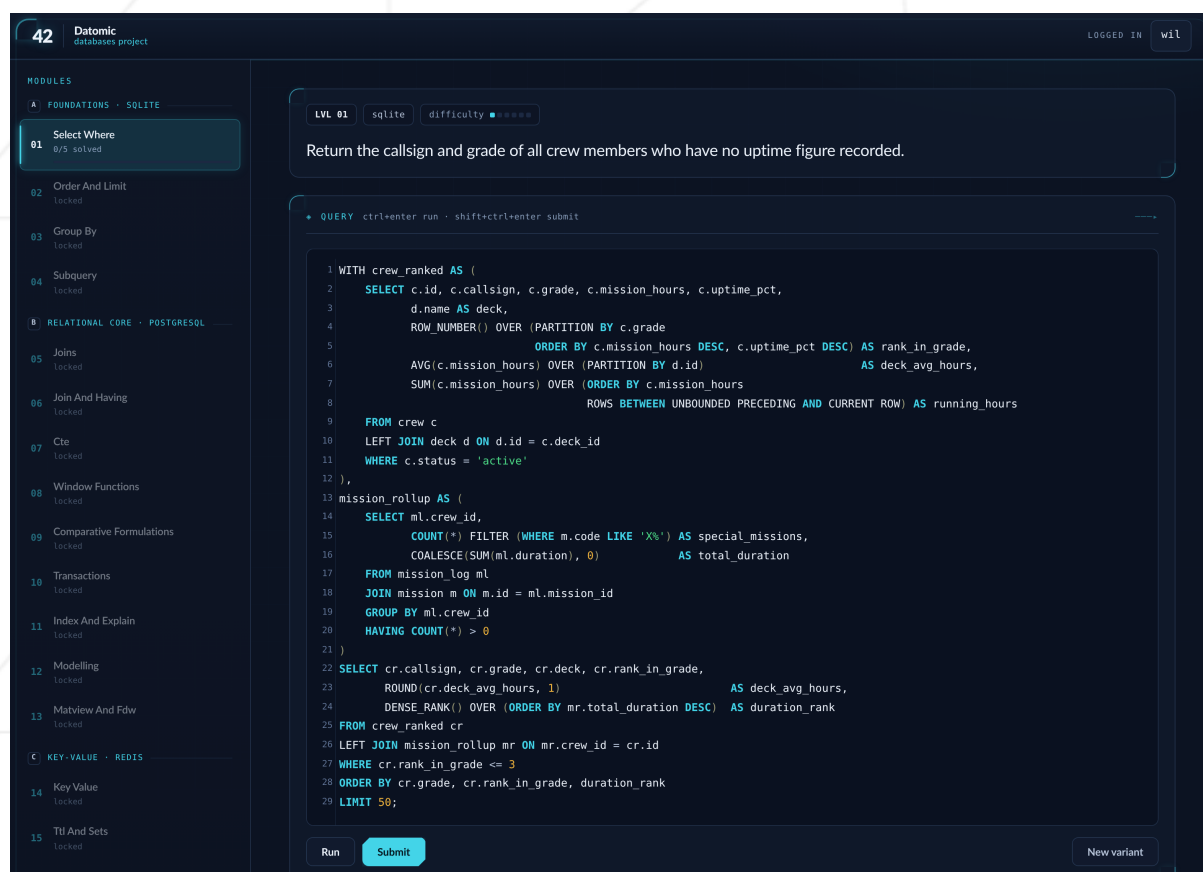


Figure III.3: The playground: a level statement above the query editor. Run shows the rows; Submit grades them against a hidden reference.

III.5 Your data

The station data is the same shape for everyone. Only the surface labels change: callsigns, module, squad and deck names, and the dates are shifted, all derived from your login. Keys, counts and metric values are identical on every VM.

Two columns are easy to confuse, so the statements always name them apart:

- **cumulative mission hours** is `crew.mission_hours`, a running total carried on the crew row.
- **logged hours** is `mission_log.hours`, one row per logged mission, so a crew member has as many of those as they have entries in the log, and none at all if they never logged one.

The *Your data* panel under the editor lists what the current engine holds: tables and columns for the SQL engines, node labels and relationships for the graph, key namespaces for the key-value store. Use it rather than guessing.

Chapter IV

Mandatory part

The mandatory part is the twenty training levels, grouped into modules by engine. You clear a level by solving one variant of it, and the bank stays open for as many more as you want. Nothing here is handed in and nothing is graded on its own: the training is where you build the fluency and the judgement, and the live review is where you show them (see the Review chapter). The modules below say what ground each one covers; the exact problems are for you to meet in the playground.

IV.1 Module A, SQLite: SQL foundations

Four levels of plain SQL to sharpen the basic reflexes: selecting and filtering rows, ordering and trimming a result, grouping rows to aggregate them, and comparing a single row against an aggregate of the others. Nothing exotic here, but the habits you build carry through the whole project.

IV.2 Module B, PostgreSQL: the relational core

The longest module and the heart of the project, nine levels that move from writing correct queries to writing queries that stay correct and fast as the data grows. You will join tables, express a query in named stages, rank and compare rows within groups, keep a multi-step change atomic, make the planner earn its keep, and reshape a schema while defending the trade-offs you made.

IV.3 Module C, Redis: key-value

Two levels on the key-value model, where the answer comes from the shape of your keys and the right primitive rather than from a query language.

IV.4 Module D, ClickHouse: columnar

Two levels on a columnar engine built to scan large event and telemetry tables, where you summarise a volume of rows that would make a row store sweat.

IV.5 Module E, Neo4j: graph

Two levels on a graph engine, where relationships are first-class and a traversal does the work a tall stack of joins would struggle with.

IV.6 Module F, DuckDB: federation

One closing level that answers a single question from two engines at once, without copying either source into the other. It is the whole point of the project in a single query.

Chapter V

Review

The project is validated in a live review with a peer, on your own VM. There is nothing to hand in: no git repository to push, no take-home defence, no slides. The whole project lives inside the station, so you sit down at it, in front of the reviewer, and work.

V.1 How it runs

The review is built around the review exam in the playground. It draws a short sequence of problems across the modules, one per slot, ordered from the easiest to the hardest, and served one at a time. You will not have seen these exact statements during training: they are a separate bank, so the review cannot be passed by memorising a solved variant.



Each slot draws from around ten forms of its problem, and your data is renamed for every login on top of that, so nobody hands you a solved exam. Do not read more into that than it says: the forms of one slot are the same problem with a different grouping, threshold or metric, so learning one of them by heart buys you nothing on the next. The playground is where you discover the SQL; the exam is where you show, live, that you understand it.

The exam gives you the same editor as the training levels: **Run** executes without grading so you can look at your rows, **Submit** is the one that counts, and only the current problem accepts either. **Start over** draws a fresh exam, and an exam you left unfinished is still there when you come back.

You solve each exam problem live, in front of the reviewer, and you spend about five minutes on each one. Those five minutes are not only for writing the query. As you work through a problem, you explain out loud:

- what the query returns, and why the result is correct,
- which engine the problem belongs to, and why that one fits,
- what the common wrong answer looks like, and how yours avoids it.

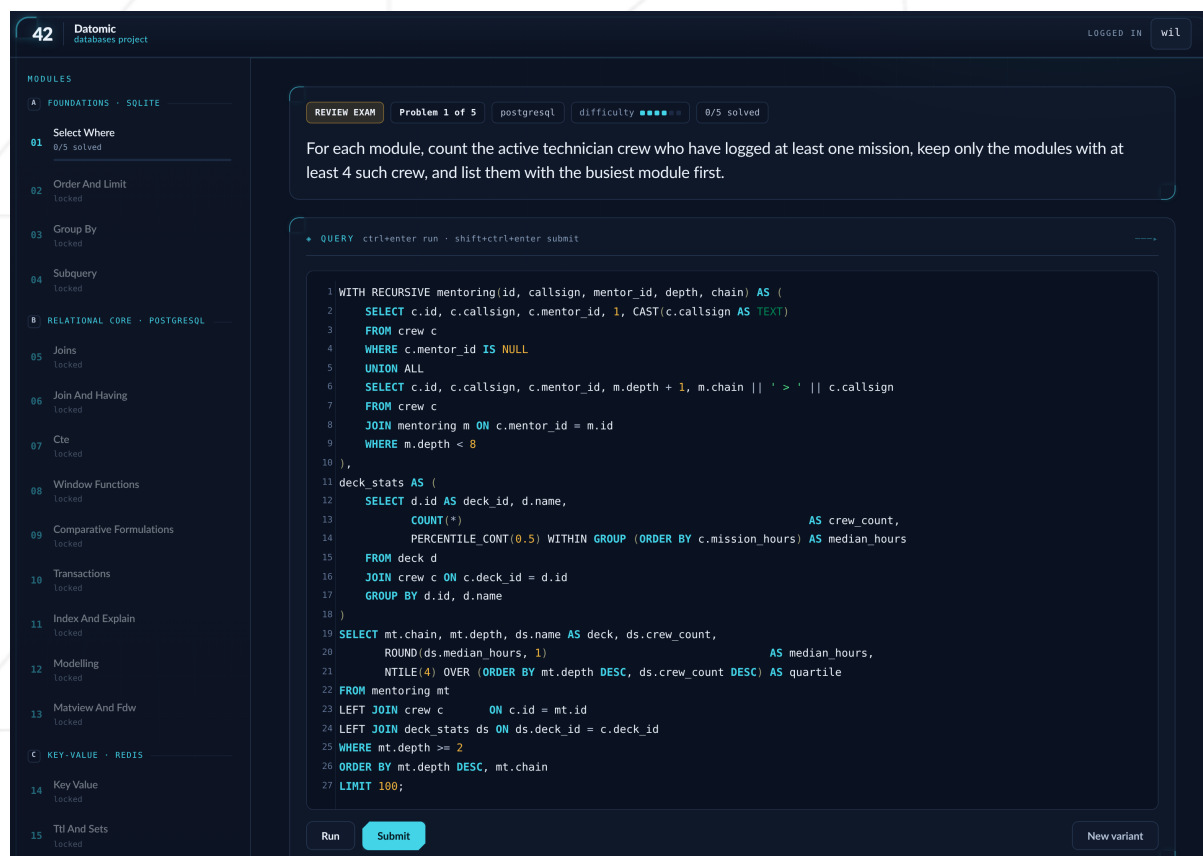


Figure V.1: The review exam: problems drawn one at a time, in order of difficulty, solved and explained in front of the reviewer.

V.2 What is graded

Producing the right rows is not enough on its own, and knowing how to write the queries is not what validates the project. There is rarely a single correct query, so the reviewer does not check yours against a key: they ask open questions and make you justify your choices, why this approach and not another, what the trade-offs are, and when a different tool would win. A query that runs but that you cannot defend does not pass, and the reviewer may bend a statement on the spot to see how far your understanding goes.

What is graded is judgement as much as output: can you say which tool fits which problem, and why federating beats duplicating. Sit the exam a few times on your own before the review, so the clock is not a surprise.